AICTE VERIFIED STUDENT REGISTER

Security, Privacy, Testing and Acceptance Criteria

Practical safety controls and evidence required before describing the prototype as demo-ready.

Document: 08-SEC-QA Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Security posture and intended use

This is a learning/demo prototype using synthetic data. It must not be used to store real government/student records or presented as production-ready until a separate security, legal, operations and integration review is completed.

2. Minimum control requirements

Area	Required control	Evidence to collect
Passwords	Use a suitable password hash; no plaintext or reversible password storage.	Inspect model, seed scripts and logs; test wrong-password response.
Sessions	HttpOnly cookie, suitable SameSite, Secure under HTTPS, expiration, session regeneration and server-side invalidation.	Test login, logout, expiry and disabled account.
CSRF	Protect cookie-authenticated state-changing requests.	Document chosen maintained mechanism and test rejection of missing/invalid token.
Authorization	Backend checks role and institution/region/student scope on every protected route.	Automated 401/403/cross-scope tests.
Public endpoints	Rate-limit login and QR verification; no open student-search endpoint.	Test repeated requests and anonymous QR output.
CSV uploads	Allow-list file types, size/row bounds, safe parser, safe temp storage and cleanup.	Test malformed/oversized file and path/name manipulation.
Secrets	Use .env in local development, provide .env.example without values, exclude .env from Git.	Secret scan/manual repository review.
Sensitive data	Use synthetic identifiers; mask fields in UI/logs/exports.	Inspect API output and logs for raw IDs/credentials.
Database	Use least-privilege credentials; transaction-capable deployment for commit; validate indexes.	Run migration/index checks and failure/rollback tests.
Audit	Append-oriented application flow, previousHash/hash checks, restricted access and reasoned events.	Alter fixture in an isolated test and confirm detection.
Errors	Do not return stack traces, passwords, session IDs or raw internal exception objects.	Force invalid and server-error cases.
Dependencies	Use locked dependency versions and inspect vulnerability/audit output before demo.	Save command output and document unresolved advisories.

3. Data-minimisation rules

- Use fully synthetic student data generated for the demo. Use fake names and artificial IDs clearly marked as demo values.
- Do not include real APAAR or Aadhaar numbers, real DOBs, production credentials or private government documents in source control, prompts, screenshots or logs.
- Store only fields needed for the demo workflow. Do not collect phone/email unless a stated synthetic scenario requires it.
- Public QR result must not permit enumerating students by sequential IDs. Use a random opaque credential reference.
- Consent must bind to purpose, fields and expiry; server checks the grant on every read, and revocation blocks later access.
- A consent UI is not a legal compliance certification. Confirm applicable privacy/legal requirements with a qualified owner before real deployment.

4. Audit-chain integrity: honest boundaries

A hash chain can reveal an accidental or uncoordinated change that breaks the chain. It does not make MongoDB immutable, and it is not sufficient protection against an attacker who can rewrite all relevant records and control the verification keys/checkpoints. The demo should claim only that a specific alteration test was detected. Production protection requires separately protected signing keys/checkpoints and independent controls.

5. QR and credential boundaries

- URL-based QR: enables lookup/status demonstration; it does not by itself prove issuer authenticity.
- Mock status endpoint: proves only the demo app state, not real government records.
- Cryptographically signed credential: deferred unless a reviewed signing/verification design and protected key handling are implemented.
- Revocation status: the verifier must check current persisted demo status; a screenshot or old cached status is not proof of current validity.

6. Required automated tests

Test group	Minimum cases	Pass condition
Auth	Valid/invalid login; disabled user; expiry; logout; missing CSRF.	Only valid, active and protected requests proceed.
Authorization	Operator tries certify; College A requests College B; regional officer accesses outside region; student accesses other student's consent.	All unauthorised attempts fail and do not mutate data.
Upload	Bad MIME/extension, malformed CSV, missing columns, large file, duplicate rows.	Safe rejection or row issues; no official writes before commit.
Validation	One test per L1-L7, plus warning/pending semantics.	Expected issue code/severity and batch outcome.
Seat transaction	At capacity, over capacity, concurrent last seat, forced transaction failure.	No over-capacity state; no partial mutation.
Batch workflow	Certify with errors; return for correction; double-submit; stale policy.	State transitions follow contract and remain idempotent.
Conflict	Transfer, unresolved, justified case, double resolution.	Correct case status/history/seat effects.
Consent	Approve, deny, expiry, revoke, wrong student, field tampering.	Only current, approved fields returned.
QR	Valid demo, revoked demo, malformed/altered reference, anonymous enumeration attempt.	Minimal status response and correct state.
Audit	Normal chain; modify one stored event; access restricted.	Integrity mismatch reported without false success.
UI	Loading, empty, error, keyboard navigation, clear labels.	No dead buttons; actions have explicit feedback.
Setup	Fresh clone, env setup, DB setup, seed, run and tests.	Another developer can follow README with no hidden secrets.

7. Release/demo acceptance gate

- All critical flows in document 01 work end-to-end with seeded synthetic data.
- All role buttons in document 04 map to APIs and permissions; no fake/dead controls are shown.
- Every protected API has positive and negative permission tests.
- Seat ledger invariants pass under concurrency and rollback tests.
- Mock external adapters are labelled in code, UI and demo narration.
- Unverified performance targets are called targets, not measured achievements.
- README explains MongoDB topology, seed accounts, environment variables, startup and tests.
- Known issues and out-of-scope production requirements are stated in the demo materials.

8. Never make these claims without evidence

- “Connected to real APAAR/DigiLocker/AICTE APIs.”
- “Fraud detection accuracy is X%” when only synthetic anomalies were planted.
- “100% tamper-proof,” “immutable,” or “blockchain-equivalent.”
- “Handles lakhs of records in production” without reproducible load tests and deployment specifications.
- “Fully compliant with all data-protection law” without a legal/compliance review.
- “Signed credential” when QR only contains a URL or ordinary identifier.